

JARVIS Desktop Assistant — Python Code

```
import speech_recognition as sr
import pyttsx3
import webbrowser
import datetime
import os
import platform
import subprocess
import urllib.parse
import pyautogui

# =====
# JARVIS - SINGLE FILE DESKTOP ASSISTANT
# =====

class Jarvis:

    def __init__(self):
        self.engine = pyttsx3.init()

        voices = self.engine.getProperty("voices")
        if voices:
            self.engine.setProperty("voice", voices[0].id)

        self.engine.setProperty("rate", 175)
        self.engine.setProperty("volume", 1.0)

        self.recognizer = sr.Recognizer()
        self.recognizer.energy_threshold = 300
        self.recognizer.dynamic_energy_threshold = True

        self.os_name = platform.system()

    def speak(self, text):
        print("JARVIS:", text)
        self.engine.say(text)
        self.engine.runAndWait()

    def listen(self):
        try:
            with sr.Microphone() as source:
                print("\nListening...")

                self.recognizer.adjust_for_ambient_noise(
                    source, duration=0.5
                )

                audio = self.recognizer.listen(
                    source, timeout=5, phrase_time_limit=8
                )

                print("Recognizing...")

                command = self.recognizer.recognize_google(
                    audio, language="en-IN"
                )

                print("YOU:", command)
                return command.lower()

        except sr.WaitTimeoutError:
            return ""

        except sr.UnknownValueError:
            print("I couldn't understand that.")
            return ""

        except sr.RequestError:
            self.speak(
                "Speech recognition service is unavailable. "
                "Please check your internet connection."
            )
            return ""

        except Exception as e:
            print("Microphone error:", e)
            return ""

    def open_website(self, url):
        webbrowser.open(url)

    def google_search(self, query):
```

```

        encoded = urllib.parse.quote(query)
        webbrowser.open(
            "https://www.google.com/search?q=" + encoded
        )

def youtube_search(self, query):
    encoded = urllib.parse.quote(query)
    webbrowser.open(
        "https://www.youtube.com/results?search_query=" + encoded
    )

def open_app(self, app):
    try:
        if self.os_name == "Windows":

            applications = {
                "notepad": "notepad.exe",
                "calculator": "calc.exe",
                "paint": "mspaint.exe",
                "explorer": "explorer.exe",
                "command prompt": "cmd.exe"
            }

            if app in applications:
                subprocess.Popen(applications[app])
                return True

            if app == "downloads":
                os.startfile(
                    os.path.join(os.path.expanduser("~"), "Downloads")
                )
                return True

            if app == "documents":
                os.startfile(
                    os.path.join(os.path.expanduser("~"), "Documents")
                )
                return True

            if app == "desktop":
                os.startfile(
                    os.path.join(os.path.expanduser("~"), "Desktop")
                )
                return True

        elif self.os_name == "Darwin":

            if app == "calculator":
                subprocess.Popen(["open", "-a", "Calculator"])
                return True

            if app == "notepad":
                subprocess.Popen(["open", "-a", "TextEdit"])
                return True

            if app == "downloads":
                subprocess.Popen([
                    "open", os.path.expanduser("~/Downloads")
                ])
                return True

        else:

            if app == "calculator":
                subprocess.Popen(["gnome-calculator"])
                return True

            if app == "notepad":
                subprocess.Popen(["gedit"])
                return True

            if app == "downloads":
                subprocess.Popen([
                    "xdg-open",
                    os.path.expanduser("~/Downloads")
                ])
                return True

    except Exception as e:
        print("Application error:", e)

    return False

def type_text(self, text):
    pyautogui.write(text, interval=0.03)

def calculate(self, expression):

```

```

try:
    allowed = "0123456789+*/().% "

    if not all(c in allowed for c in expression):
        return None

    return eval(
        expression,
        {"__builtins__": None},
        {}
    )

except Exception:
    return None

def tell_time(self):
    now = datetime.datetime.now()
    self.speak(
        f"The time is {now.strftime('%I:%M %p')}}"
    )

def tell_date(self):
    now = datetime.datetime.now()
    self.speak(
        f"Today is {now.strftime('%A, %d %B %Y')}}"
    )

def system_info(self):
    system = platform.system()
    version = platform.version()
    machine = platform.machine()

    self.speak(
        f"You are running {system}. "
        f"Your system architecture is {machine}."
    )

    print("System:", system)
    print("Version:", version)
    print("Architecture:", machine)

def joke(self):
    import random

    jokes = [
        "Why did the computer go to the doctor? "
        "Because it had a virus.",
        "Why do programmers prefer dark mode? "
        "Because light attracts bugs.",
        "There are only 10 types of people in the world. "
        "Those who understand binary and those who don't."
    ]

    self.speak(random.choice(jokes))

def process_command(self, command):
    if not command:
        return True

    if any(word in command for word in [
        "exit", "quit", "goodbye", "shutdown jarvis", "stop"
    ]):
        self.speak("Goodbye. Have a great day.")
        return False

    if any(word in command for word in [
        "hello", "hi jarvis", "hey jarvis"
    ]):
        self.speak(
            "Hello. I am Jarvis. How can I help you?"
        )
        return True

    if "time" in command:
        self.tell_time()
        return True

    if "date" in command or "today" in command:
        self.tell_date()
        return True

    if "joke" in command:
        self.joke()
        return True

    if (

```

```

        "system information" in command
        or "system info" in command
        or "computer information" in command
    ):
        self.system_info()
        return True

    if command.startswith("search google for"):
        query = command.replace(
            "search google for", "", 1
        ).strip()

        if query:
            self.speak(
                f"Searching Google for {query}"
            )
            self.google_search(query)

        return True

    if command.startswith("google"):
        query = command.replace("google", "", 1).strip()

        if query:
            self.speak(
                f"Searching Google for {query}"
            )
            self.google_search(query)

        return True

    if command.startswith("play"):
        query = command.replace("play", "", 1).strip()

        if query:
            self.speak(
                f"Searching YouTube for {query}"
            )
            self.youtube_search(query)

        return True

    if "youtube" in command:
        query = command.replace("youtube", "").strip()

        if query:
            self.youtube_search(query)
        else:
            self.open_website(
                "https://www.youtube.com"
            )

        return True

    websites = {
        "open google": "https://www.google.com",
        "open youtube": "https://www.youtube.com",
        "open github": "https://github.com",
        "open gmail": "https://mail.google.com",
        "open facebook": "https://www.facebook.com",
        "open instagram": "https://www.instagram.com",
        "open wikipedia": "https://www.wikipedia.org"
    }

    if command in websites:
        self.open_website(websites[command])
        self.speak("Opening it now.")
        return True

    applications = [
        "notepad",
        "calculator",
        "paint",
        "explorer",
        "command prompt",
        "downloads",
        "documents",
        "desktop"
    ]

    for app in applications:
        if command == f"open {app}" or command == app:
            success = self.open_app(app)

            if success:
                self.speak(f"Opening {app}.")

```

```

        else:
            self.speak(
                f"I could not open {app}."
            )

        return True

    if command.startswith("type"):
        text = command.replace("type", "", 1).strip()

        if text:
            self.speak("Typing now.")
            self.type_text(text)

        return True

    if command.startswith("calculate"):
        expression = command.replace(
            "calculate", "", 1
        ).strip()

        result = self.calculate(expression)

        if result is not None:
            self.speak(
                f"The answer is {result}"
            )
        else:
            self.speak(
                "I could not calculate that."
            )

        return True

    if command.startswith("open website"):
        site = command.replace(
            "open website", "", 1
        ).strip()

        if site:
            if not site.startswith("http"):
                site = "https://" + site

            self.open_website(site)
            self.speak(
                "Opening the website."
            )

        return True

    self.speak(
        "I don't know that command yet. "
        "I can search Google if you ask me to."
    )

    return True

def run(self):

    print("=" * 60)
    print("                JARVIS DESKTOP ASSISTANT")
    print("=" * 60)

    print("\nExample commands:")
    print(" Hello Jarvis")
    print(" What is the time")
    print(" What is today's date")
    print(" Open Google")
    print(" Open YouTube")
    print(" Play relaxing music")
    print(" Search Google for Python")
    print(" Open Notepad")
    print(" Open Calculator")
    print(" Open Downloads")
    print(" Calculate 25 * 4")
    print(" Type hello world")
    print(" Tell me a joke")
    print(" System information")
    print(" Exit")
    print("=" * 60)

    self.speak(
        "Jarvis is online. How can I help you?"
    )

    while True:

```

```

        command = self.listen()

        if not command:
            try:
                command = input(
                    "\nType command "
                    "(or press Enter to listen): "
                ).lower().strip()
            except KeyboardInterrupt:
                break

            if not command:
                continue

        if not self.process_command(command):
            break

if __name__ == "__main__":
    try:
        jarvis = Jarvis()
        jarvis.run()

    except KeyboardInterrupt:
        print("\nJarvis stopped.")

    except Exception as error:
        print("\nJarvis encountered an error:")
        print(error)

```